ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INENIERÍA DE SISTEMAS
APLICACIONES MÓVILES

Integrantes: Javier Angulo
Jotcelyn Godoy
Nahomy Llumiquinga
Fecha: 14/04/2026
Grupo: 1
Taller: Arquitectura y Evolución del Desarrollo Móvil

Parte 1: El Viaje Arquitectónico del Desarrollo Móvil (Desarrollo)

1. Desarrollo Nativo Puro (iOS / Android)

El desarrollo nativo representa la forma más tradicional y directa de crear aplicaciones móviles, donde se utilizan los lenguajes y herramientas oficiales de cada plataforma. En el caso de Android, se emplean principalmente Java y Kotlin, mientras que en iOS se utilizan Objective-C y Swift. Estas aplicaciones interactúan directamente con el sistema operativo mediante APIs nativas, lo que permite un alto rendimiento, acceso completo al hardware y una integración óptima con el ecosistema del dispositivo. Si nos referimos a la arquitectura, el desarrollo nativo se basa en patrones como MVC, MVP y MVVM, junto con frameworks propios como Android SDK y UIKit/SwiftUI en iOS. Su principal ventaja es la eficiencia y la experiencia de usuario fluida, aunque presenta el inconveniente de requerir el desarrollo y mantenimiento de dos bases de código separadas.

2. La Era del WebView / Híbridos (Cordova, PhoneGap, Ionic inicial)

Con el objetivo de reducir costos y tiempos de desarrollo, surgieron las aplicaciones híbridas basadas en WebView, que permiten desarrollar aplicaciones móviles utilizando tecnologías web como HTML, CSS y JavaScript. Frameworks como Apache Cordova, PhoneGap e Ionic (en sus primeras versiones) encapsulan una aplicación web dentro de un contenedor nativo.

El funcionamiento se basa en un componente WebView que actúa como un navegador embebido dentro de la aplicación, renderizando la interfaz mediante tecnologías web. Para acceder a funcionalidades del dispositivo (cámara, GPS, etc.), se utilizan plugins que actúan como intermediarios entre JavaScript y el sistema operativo.

3. El Puente Nativo-JavaScript y otros puentes

La siguiente evolución fue el uso de arquitecturas basadas en un “puente” entre código nativo y lógica en JavaScript. Frameworks como React Native introdujeron este modelo, donde la lógica de la aplicación se escribe en JavaScript, pero los componentes visuales se renderizan utilizando elementos nativos reales. Este puente permite la comunicación entre ambos mundos, aunque introduce cierta latencia y complejidad en la sincronización de datos. A pesar de esto, ofrece una mejora significativa en rendimiento comparado con WebView, además de permitir reutilización de código entre plataformas. Existen Otras tecnologías como Xamarin (C#) o NativeScript que también implementan este concepto, cada una con diferentes enfoques y actualmente existen evolucion de esta.

4. El Motor de Renderizado Propio

Frameworks modernos como Flutter representan un cambio significativo en la arquitectura móvil. En lugar de depender de componentes nativos o WebView, utilizan un motor de renderizado propio (como Skia) para dibujar toda la interfaz desde cero. Flutter utiliza el lenguaje Dart y permite construir interfaces altamente personalizables y consistentes entre plataformas. Al eliminar la dependencia del sistema operativo para el renderizado, se logra mayor control, mejor rendimiento y menos inconsistencias visuales.

5. El Futuro: Foldables, AR y Wearables

La evolución del desarrollo móvil actual ha dado origen a tecnologías emergentes. Las pantallas plegables (Foldables) son dispositivos que permiten cambiar el tamaño y formato de la interfaz dinámicamente, obligando a las aplicaciones a adaptarse en tiempo real. Los dispositivos wearables, como relojes inteligentes o pulseras, son equipos portátiles con pantallas pequeñas y funciones específicas, lo que requiere interfaces simplificadas, rápidas y altamente contextuales. Por otro lado, la realidad aumentada (AR) permite superponer información digital sobre el entorno físico del usuario mediante la cámara del dispositivo, creando experiencias inmersivas e interactivas.

En este contexto, los desarrolladores deben implementar interfaces adaptativas, es decir, interfaces que se ajustan automáticamente a distintos tamaños de pantalla y dispositivos, así como diseño responsivo avanzado, que no solo reorganiza elementos visuales, sino que también modifica funcionalidades según el contexto de uso. Además, surgen nuevas formas de interacción como el uso de gestos táctiles, comandos de voz y sensores del dispositivo (como acelerómetro o GPS), que permiten una interacción más natural.

Parte 2: Taller Práctico - Análisis Crítico de Aplicaciones

- **Caso de Estudio A:** Pinterest (Construido con Nativo + Texture/AsyncDisplayKit)
- **Caso de Estudio B:** Pokémon GO (Construido con Unity)

1. Ingeniería Inversa de la Decisión

¿Por qué Pinterest apostó por una arquitectura Nativa potenciada con Texture?

Pinterest es una aplicación centrada en la visualización masiva de contenido (imágenes, textos, miniaturas y grids infinitos). Su experiencia depende casi completamente de que el scroll sea fluido y estable, incluso en dispositivos de gama media o baja.
La decisión de usar desarrollo nativo (Swift/Objective-C en iOS y Kotlin/Java en Android) junto con Texture se explica por razones técnicas:

- El contenido visual es extremadamente pesado, ya que requiere carga de imágenes, cálculo dinámico de tamaños, sombras y renderizado constante.
- Texture permite que gran parte del trabajo (layout, medición de texto, pre-renderizado de nodos) se ejecute en hilos secundarios, liberando el Main Thread.
- La arquitectura de UI nativa permite optimización profunda del uso de memoria, reciclaje de componentes y control fino de cachés.

¿Por qué Pokémon GO apostó por Unity en lugar de un desarrollo puramente nativo?

Pokémon GO no es una app convencional de UI; es un sistema híbrido entre videojuego, geolocalización y realidad aumentada.

Unity fue una elección estratégica porque:

- Pokémon GO requiere renderizado 3D constante y animaciones complejas.
- Unity ofrece un motor gráfico listo para manejar iluminación, físicas, modelos 3D y efectos visuales.
- La lógica del juego funciona mejor bajo un modelo de Game Loop, donde todo se actualiza continuamente.
- Unity facilita desplegar un mismo proyecto para múltiples plataformas con consistencia visual.
- Niantic ya tenía experiencia previa con Unity gracias a su juego anterior Ingress, por lo que reutilizar infraestructura era viable.

2. Análisis de Rendimiento y Arquitectura

¿Qué desafíos arquitectónicos enfrenta Pinterest al manejar renderizado masivo y memoria?

Pinterest enfrenta un problema clásico de escalabilidad visual: mostrar miles de imágenes y elementos sin que el dispositivo colapse por RAM o procesamiento.

Los principales desafíos son:

- Carga progresiva de imágenes: el sistema debe mostrar contenido rápidamente usando placeholders mientras descarga imágenes reales.
- Layout dinámico: el grid depende de tamaños variables de imágenes, por lo que debe calcular dimensiones antes de renderizar.
- Gestión agresiva de memoria: al ser un scroll infinito, la app debe reciclar nodos y liberar recursos inmediatamente.
- Concurrencia controlada: mover operaciones fuera del hilo principal mejora rendimiento, pero introduce complejidad por sincronización de estados y race conditions.

¿Qué desafíos enfrenta Pokémon GO al sincronizar GPS con renderizado 3D sin generar desfase?

Pokémon GO enfrenta un desafío arquitectónico más complejo porque combina: datos físicos (GPS, acelerómetro, giroscopio), renderizado 3D, y lógica de juego conectada a servidores.

Los principales problemas son:

- Latencia del GPS: las coordenadas no llegan en tiempo real continuo, sino en intervalos con error y ruido.
- Desincronización entre GPS y FPS: el GPS actualiza cada cientos de milisegundos, mientras el renderizado ocurre a 30–60 FPS.
- Interpolación de movimiento: el sistema debe “inventar” movimiento fluido entre puntos GPS para evitar saltos bruscos.
- Carga de GPU/CPU: renderizar mundo 3D y procesar sensores al mismo tiempo puede provocar caída de FPS.
- Sincronización con servidores: eventos como capturas, spawns y batallas dependen de validación online, generando retrasos adicionales.

3. **El Desafío de las Nuevas Pantallas (AR Glasses / Lentes Inteligentes)**

Si mañana Apple y Samsung lanzan lentes de AR como estándar, ¿cuál arquitectura se adaptaría mejor Nativo optimizado para UI o Unity optimizado para 3D?

En lentes inteligentes, la arquitectura ganadora dependerá del tipo de aplicación dominante: consumo de información o interacción inmersiva.

- **Pinterest (Nativo + Texture) en lentes AR**

Su enfoque sería ideal para experiencias como: feeds flotantes, dashboards, notificaciones, interfaces ligeras tipo "tarjetas".

Su ventaja es que el sistema: consume menos batería, es más eficiente en memoria, y permite UI rápida sin renderizado constante.

- **Pokémon GO (Unity) en lentes AR**

Unity se adapta mejor a lentes inteligentes cuando se necesita: posicionar objetos virtuales en el mundo real, renderizar modelos 3D, detectar profundidad, mantener interacción espacial continua.

Su arquitectura es perfecta para AR real porque ya opera bajo un ciclo de actualización permanente.

Para apps basadas en información y productividad, el estándar sería Pinterest (arquitectura nativa optimizada).
Para AR inmersiva real (gaming, simulación, objetos 3D persistentes), el estándar sería Unity.

En lentes inteligentes, Unity dominará experiencias inmersivas, pero la arquitectura nativa será más común por ser más liviana y eficiente.

4. **Veredicto Crítico (Decisión de Arquitectura Realista)**

Como futuros ingenieros de software: Si estuvieran creando hoy una aplicación que requiere interactuar con mapas 3D interactivos, notificaciones de alta frecuencia y debe soportar pantallas plegables... ¿elegirían Nativo + Texture o Unity? Fundamenten su respuesta basados estrictamente en la arquitectura de la herramienta, no en gustos personales por un lenguaje.

Si se debe construir una aplicación con mapas 3D interactivos, notificaciones de alta frecuencia y soporte para pantallas plegables, la decisión depende de la arquitectura base.

Nativo + Texture es ideal para aplicaciones centradas en UI y eventos, ya que ofrece mejor integración con el sistema operativo, mejor manejo de notificaciones y mayor eficiencia en consumo de batería. Además, se adapta naturalmente a cambios de tamaño de pantalla en dispositivos plegables.

Por otro lado, Unity está basado en un Game Loop, lo cual lo hace superior para renderizado 3D continuo y mapas interactivos, manteniendo animaciones y escenas fluidas.

Si el mapa 3D es el núcleo principal de la app, la mejor opción arquitectónica es Unity. Sin embargo, para no afectar rendimiento y UX, lo más recomendable sería usar Unity solo para el módulo 3D e integrar notificaciones y servicios críticos con componentes nativos.

Entonces, ¿Qué es un aplicativo nativo y cómo se clasifican?

Un aplicativo nativo es un tipo de software que se desarrolla específicamente para ejecutarse en un sistema operativo o plataforma determinada, como Android, iOS o Windows. Este tipo de aplicación se instala directamente en el dispositivo y está diseñada utilizando herramientas y lenguajes propios de cada plataforma, lo que le permite ofrecer un funcionamiento más eficiente, mayor seguridad y una experiencia de usuario adaptada al entorno del dispositivo. Además, puede acceder a recursos del hardware como la cámara, el GPS o las notificaciones, lo que mejora sus funcionalidades y rendimiento.

En cuanto a su clasificación, los aplicativos nativos pueden organizarse principalmente de dos maneras:

1. Según la plataforma en la que funcionan:

- Aplicaciones nativas para Android
- Aplicaciones nativas para iOS
- Aplicaciones nativas para otros sistemas como Windows o macOS

2. Según su finalidad o uso:

- Aplicaciones de entretenimiento (como videojuegos)
- Aplicaciones de productividad (gestión de tareas, almacenamiento)
- Aplicaciones de comunicación (mensajería, videollamadas)
- Aplicaciones empresariales o de negocio

En general, los aplicativos nativos destacan porque están optimizados para cada dispositivo, lo que se traduce en mayor velocidad, mejor interfaz y una interacción más fluida con el usuario. Sin embargo, su desarrollo suele implicar mayores costos y tiempo en comparación con otros tipos de aplicaciones.

Bibliografía

[1] Google, *"Android Developers Documentation"*, Google, 2024. [En línea]. Disponible en: https://developer.android.com. [Accedido: 14-abr-2026].

[2] Apple Inc., *"iOS Developer Documentation"*, Apple, 2024. [En línea]. Disponible en: https://developer.apple.com. [Accedido: 14-abr-2026].

[3] Apache Software Foundation, *"Apache Cordova Documentation"*, Apache, 2023. [En línea]. Disponible en: https://cordova.apache.org. [Accedido: 14-abr-2026].

[4] Ionic, *"Ionic Framework Documentation"*, Ionic, 2024. [En línea]. Disponible en: https://ionicframework.com/docs. [Accedido: 14-abr-2026].

[5] Meta Platforms, *"React Native Documentation"*, Meta, 2024. [En línea]. Disponible en: https://reactnative.dev. [Accedido: 14-abr-2026].

[6] Microsoft, *"Xamarin Documentation"*, Microsoft, 2024. [En línea]. Disponible en: https://learn.microsoft.com/xamarin. [Accedido: 14-abr-2026].

[7] Google, *"Flutter Documentation"*, Google, 2024. [En línea]. Disponible en: https://docs.flutter.dev. [Accedido: 14-abr-2026].

[8] Google, *"ARCore Developer Guide"*, Google, 2024. [En línea]. Disponible en: https://developers.google.com/ar. [Accedido: 14-abr-2026].

[9] Apple Inc., *"ARKit Documentation"*, Apple, 2024. [En línea]. Disponible en: https://developer.apple.com/augmented-reality. [Accedido: 14-abr-2026].

[10] R. P. Singh, "¿Qué es una aplicación nativa? | Una guía completa," *Richestsoft*, Oct. 17, 2024. [En línea]. Disponible en: https://richestsoft.com/es/blog/what-is-a-native-app/. [Accedido: Abr. 14, 2026].

[11] Camp Tecnológico, "Descubre cómo Unity dio vida a Pokémon GO y aprende a usarlo en los campamentos de Camp Tecnológico," *Camp Tecnológico*, 2024. [En línea]. Disponible en: https://camptecnologico.com/unity-pokemon-go/. [Accedido: 14-abr-2026].

[12] Hacker News, "Discord: Switching from React Native to Native (Android)," *news.ycombinator.com*, dic. 2024. [En línea]. Disponible en: https://news.ycombinator.com/item?id=42201291. [Accedido: 14-abr-2026].

[13] W. Pasman, S. Persa, y F. W. Jansen, "Realistic low-latency mobile AR rendering," Delft University of Technology, UbiCom project, Delft, Países Bajos, Rep. Técnico, s.f. [En línea]. Disponible: https://twi.tudelft.nl. [Accedido: 14-abr-2026].

[14] S. G. Kim, "Augmented Reality for Mobile Devices: A Review of Technology and Applications," *Sensors*, vol. 23, n.º 21, p. 8837, oct. 2023, doi: 10.3390/s23218837. [En línea]. Disponible en: https://pmc.ncbi.nlm.nih.gov/articles/PMC10650837/. [Accedido: 14-abr-2026].

[15] Á. Herrero, "Pokémon GO tendría un sistema más real de realidad aumentada," *AreaJugones*, 2024. [En línea]. Disponible en: https://areajugones.sport.es/videojuegos/pokemon-go-tendria-un-sistema-mas-real-de-realidad-aumentada/. [Accedido: 14-abr-2026].

[16] J. García, "Niantic, la que conquistó la realidad aumentada en móviles con Pokémon Go, ya prepara sus propias gafas AR," *Xataka*, 2024. [En línea]. Disponible en: https://www.xataka.com/realidad-virtual-aumentada/niantic-que-conquisto-realidad-aumentada-moviles-pokemon-go-prepara-sus-propias-gafas-ar. [Accedido: 14-abr-2026].